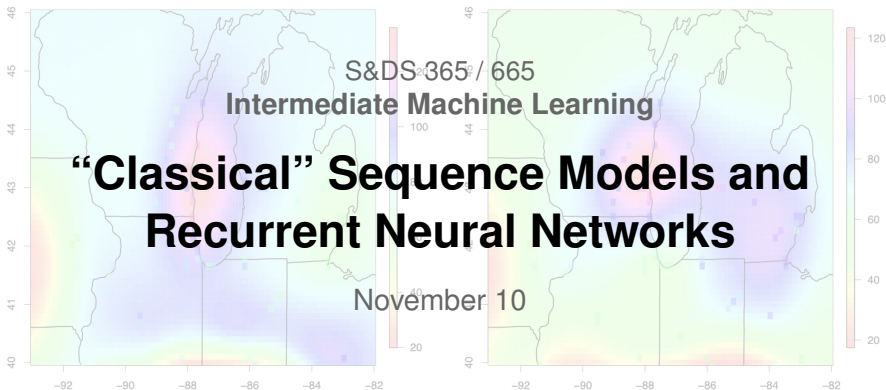




Reminders

- Final exam: Friday, Dec 12, 9am
- Assignment 4 is out, due next Thursday
 - ▶ Graph neural nets
 - ▶ Policy gradient
 - ▶ Deep Q-Learning
- Assignment 5 (last!) posted Wednesday
 - ▶ GRUs
 - ▶ Transformers

Quiz 4

Ⓜ Average Score

94%

Ⓢ High Score

100%

Ⓣ Low Score

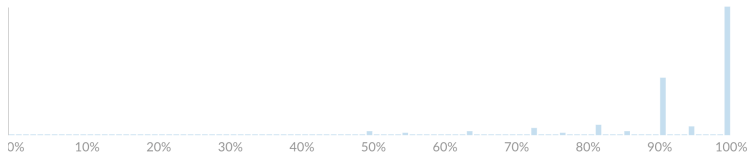
50%

Ⓢ Standard
Deviation

1.09

⌚ Average Time

23:19



For Today

- Actor-critic: quick recap
- Sequence models
 - ▶ HMMs, KFs, RNNs
 - ▶ RNN examples

Next couple weeks

- This week:
 - ▶ Classical sequence models
 - ▶ Memory in RNNs
- Next week:
 - ▶ Transformers
 - ▶ Post-processing LLMs

Recap: Actor-critic approaches

- Estimate policy and value function together
- Actor: policy used to select actions
- Critic: value function used to criticize actor
- Error signal from the critic drives all learning
- An on-policy approach



Actor-critic approaches

- After each selected action, critic evaluates new state:
 - ▶ Have things gone better or worse than expected?
- The error signal is used to update actor and value function

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t)$$

$\delta_t > 0$: action was better than expected

$\delta_t < 0$: action was worse than expected

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t)$$

Value function is updated as

$$V(\mathbf{s}_t) \leftarrow V(\mathbf{s}_t) + \alpha \delta_t$$

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t)$$

Used to update parameters of policy.

If δ_t is positive (negative), action a_t should become more (less) probable in state s_t

For example, with

$$\pi_{\theta}(a | s) = \text{Softmax}\{f_{\theta}(s, a_1), \dots, f_{\theta}(s, a_D)\}$$

parameters θ adjusted so $f_{\theta}(s_t, a_t)$ increases (decreases)

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Used to update parameters of policy.

For example, with

$$\pi_{\theta}(a | s) = \text{Softmax}\{f_{\theta}(s, a_1), \dots, f_{\theta}(s, a_D)\}$$

parameters θ adjusted so $f_{\theta}(s_t, a_t)$ increases (decreases)

$$\begin{aligned}\theta &\leftarrow \theta + \eta \delta_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \\ &= \theta + \eta \delta_t \nabla_{\theta} f_{\theta}(s_t, a_t)\end{aligned}$$

Next few classes

We'll talk about different approaches to sequence modeling (including language models), leading up to transformers.

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

Language models

- A language model is a way of assigning a probability to any sequence of words (or string of text)

$$p(w_1, \dots, w_n)$$

- By the basic rules of conditional probability we can factor this as

$$p(w_1, \dots, w_n) = p(w_1)p(w_2 | w_1) \dots p(w_n | w_1, \dots, w_{n-1})$$

Text generation

- Words (tokens) generated one-by-one
- A word (token) is chosen by sampling from a probability distribution
- Condition on previously generated text, as if “real”
- Result is purely synthetic text

Modern language models

Suppose a computer program assigns a “score” to possible next words:

$$f_{\theta}(v; \underbrace{s(w_1, \dots, w_n)}_{\text{state of word history}})$$

Modern language models

Suppose a computer program assigns a “score” to possible next words:

$$f_{\theta}(v; \underbrace{s(w_1, \dots, w_n)}_{\text{state of word history}})$$

Can convert this to a language model by the “softmax” operation:

$$p(w \mid w_1, \dots, w_n) = \frac{\exp(f_{\theta}(w; s(w_1, \dots, w_n)))}{\sum_{v \in V} \exp(f_{\theta}(v; s(w_1, \dots, w_n)))}$$

First things first

- In an LLM, the function $f_{\theta}(v; s(w_{1:n}))$ is learned on large amounts of text to predict the next word (unsupervised) using a type of deep neural network called a *transformer*
- We'll first discuss “classical” sequence models — HMMs, KFs, RNNs

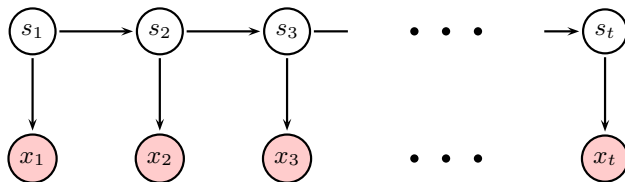
Hidden Markov Model

In an HMM, the next word is generated from a latent variable

- State is chosen stochastically (that is, probabilistically)
- Introduces dependence on earlier words that are further back than the previous time (non-Markov on observations)

Hidden Markov Model

The graphical model looks like this:



Here x_t is observable (word) at time t and s_t is unobserved state.

Hidden Markov Model

Probability of word sequence:

$$p(x_1, \dots, x_n) = \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | s_{t-1}) p(x_t | s_t)$$

Hidden Markov Model

Probability of word sequence:

$$p(x_1, \dots, x_n) = \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | s_{t-1}) p(x_t | s_t)$$

Topic models are Bayesian models with a latent variable θ representing a distribution over “topics”:

$$p(x_1, \dots, x_n) = \int p(\theta | \alpha) \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | \theta) p(x_t | s_t) d\theta$$

Word order doesn't matter

Hidden Markov Model

Probability of word sequence:

$$p(x_1, \dots, x_n) = \sum_{s_1, \dots, s_n} \prod_{t=1}^n p(s_t | s_{t-1}) p(x_t | s_t)$$

Can be efficiently computed using the “forward-backward algorithm” which is a type of dynamic program:

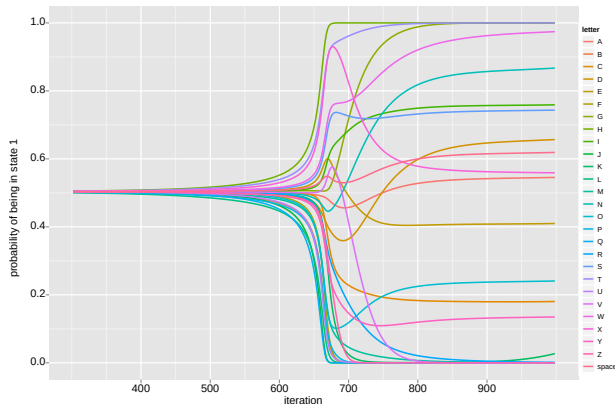
$$p(x_n | x_1, \dots, x_{n-1}) = \sum_{s_n} p(s_n | x_1, \dots, x_{n-1}) p(x_n | s_n)$$

Hidden Markov Models: Estimation

The usual algorithms for estimating the parameters are iterative:

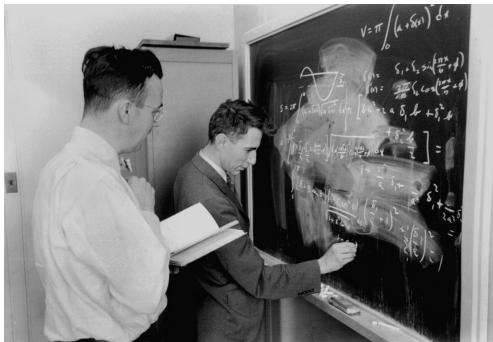
- Use the forward-backward algorithm to compute the probability that the outputs are generated from each state
- Treat the data as labeled, with these weights, and compute the relative frequencies
- Repeat
- Maximum likelihood (or MAP) estimation

Hidden Markov Models: Example



HMM has two states, emits one of the twenty-six letters A-Z. Trained on 20,000 characters of English text, initial parameters close to uniform. Curves show the posterior probability $\mathbb{P}(\text{state} = 1 \mid \text{letter}, \mathcal{D})$ of the probability each letter is generated from state one. “phase transition” at around 650 iterations.

Claude Shannon



Model underlying Kalman filters

A Gaussian HMM:

$$S_t | S_{t-1} \sim N(AS_{t-1}, \Gamma)$$

$$X_t | S_t \sim N(BS_t, \Sigma)$$

where A and B are matrices for the means; Γ and Σ for the covariances. Everything is linear.

- State is distributed (real vector)
- State evolves stochastically
- Schur complements and forward-backward used to compute conditional probabilities; similar to Gaussian processes

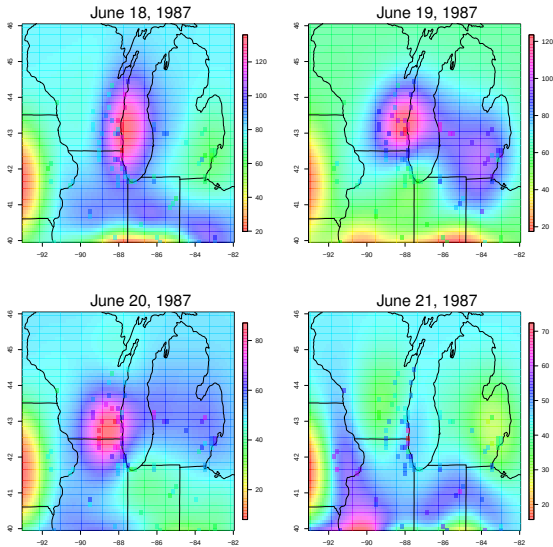
Example: Ozone

Data are daily maximum 8-hour ozone concentrations (in parts-per-billion) at 153 sites in the US midwest near Lake Michigan, for 89 days during the summer of 1987.

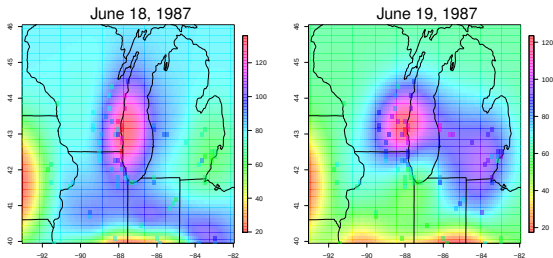
- `lon.lat`: longitudes and latitudes of the 153 stations;
- `y`: measurements at each stations on each day;
- `station.id` and `dates`.

Can use Kalman filtering/smoothing to infer ozone concentrations of the area

Kalman filtering: Example

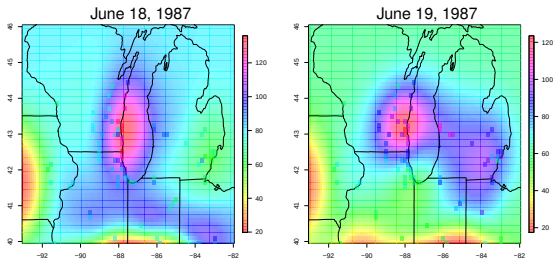


Kalman filtering: Example



- In this example, $A = I$ (just add noise)
- $B_j =$ “average over locations near j ”.

Kalman filtering: Example



- The Kalman filter is a generative model
- You can “synthesize” ozone levels on a given day.
- Could use kernel smoothing (a discriminative model) or a Gaussian process

Kalman filtering: Discrete data

We can also work with state space models for discrete data, like documents.

$$S_t \mid S_{t-1} \sim N(AS_{t-1}, \Gamma)$$

$$W_t \mid S_t \sim \text{Softmax}(BS_t)$$

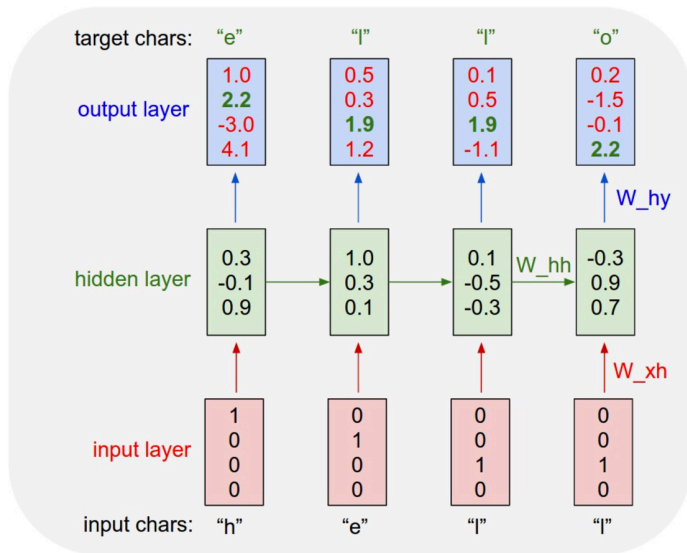
where

$$\text{Softmax}(BS_t)_j = \frac{\exp(B_j^T S_t)}{\sum_k \exp(B_k^T S_t)}$$

Next: Recurrent neural networks

- Can be thought of as a type of language model
- Similar to hidden Markov models and Kalman filters, but the hidden layer is not stochastic
- The state is distributed (a vector), as in Kalman filters, not categorical as for HMMs

RNNs



RNNs

This means

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

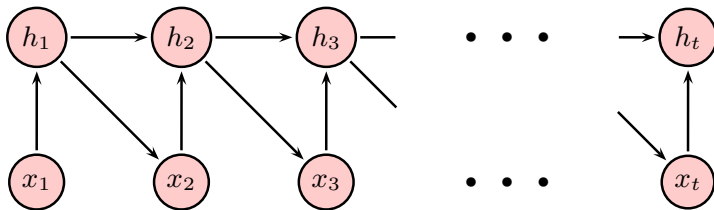
$$y_t = W_{hy}h_t$$

$$x_{t+1} \mid y_t \sim \text{Multinomial}(\pi(y_t))$$

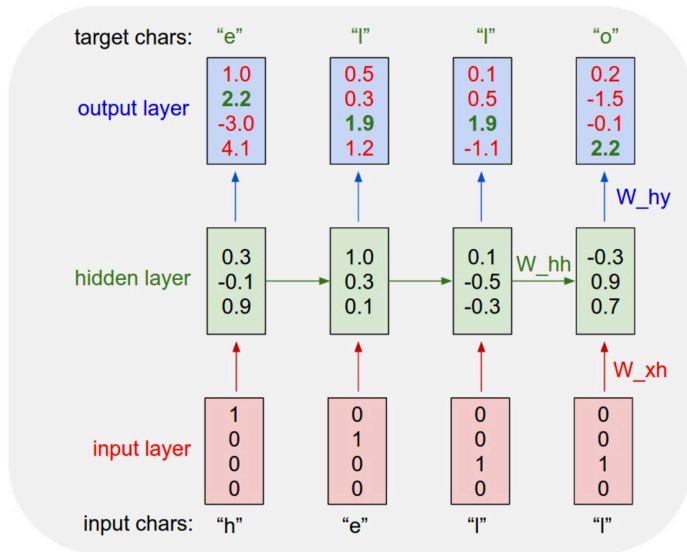
where $\pi(\cdot)$ is the soft-max function.

In this illustration, x_t is the “1-hot” representation of a character, $W_{xh} \in \mathbb{R}^{3 \times 4}$, $W_{hh} \in \mathbb{R}^{3 \times 3}$ and $W_{hy} \in \mathbb{R}^{4 \times 3}$.

RNN graphical structure



RNN architecture



Summary

- HMMs and Kalman Filters are classical models for sequences
- HMMs have discrete states, KFs have distributed states and Gaussian transitions/emissions
- Recurrent neural networks are also used for sequential data, like language
- As usual, linear mappings followed by activation functions.
- RNNs are unsupervised, generative models, able to generate realistic looking sequences when sufficiently well trained.
- LSTMs and GRUs model longer range dependencies (next time)